

BLOC 4

Maintenir l'application logicielle en condition opérationnelle

Dossier jury - RNCP39583 Expert en développement logiciel

Projet Alcide

Application web de génération d'entraînements et programmes sportifs personnalisés. Candidat : Kevin. Version de référence : 0.13.0-rc.8. État : 18 août 2026.

Objet du dossier

Ce dossier présente le monitoring, le traitement des anomalies et la maintenance de l'application. Il relie les mécanismes versionnés aux traces d'exécution conservées dans GitHub.

Périmètre des preuves

Les références GitHub citées correspondent à des runs, artefacts, issues et pull requests datés. Les simulations déclarées sont séparées des incidents réellement détectés en production.

Livrable écrit individuel - 20 pages maximum

1. Exigences et périmètre MCO

Le Bloc 4 évalue la maintenance d'un logiciel développé pendant la formation : mises à jour, supervision, consignation et correction des anomalies, évolutions, versions et collaboration avec le support.

Compétence	Attendu évalué	Réponse Alcide
C4.1.1	Mettre à jour les dépendances de manière sécurisée.	Dependabot, audit, CI et procédure de qualification/rollback.
C4.1.2*	Supervision, indicateurs, sondes, seuils et signalement.	Readiness API, healthcheck Web, monitoring GitHub horaire, artefact et issue automatique.
C4.2.1*	Collecter, consigner et analyser une anomalie.	Processus d'incident, templates GitHub et fiches BUG documentées.
C4.2.2	Corriger et déployer via l'intégration et le déploiement continu.	PR checklist, CI/CD, smoke tests et rollback Vercel documentés.
C4.3.2*	Établir un journal des versions et des correctifs.	CHANGELOG versionné, matrice de traçabilité maintenance.
C4.3.3	Collaborer avec le support sur un problème complexe.	Cas support pilote documenté dans l'issue GitHub #12 avec diagnostic et validation.

* Compétence éliminatoire selon le règlement de certification.

Architecture de production

Composant	Rôle	Point de maintenance
Web Next.js	Interface utilisateur et OAuth	Healthcheck /api/health, erreurs d'authentification, version.
API Hono	Logique métier, IA, accès aux données	Liveness /health et readiness /health/ready.
Neon PostgreSQL	Persistance et migrations Drizzle	Disponibilité, erreurs de connexion, branche/backup avant migration.
Vercel + GitHub Actions	Déploiement et contrôles	CI, CD, smoke tests, monitoring et rollback.

2. Mises à jour des dépendances

La maintenance des dépendances couvre les packages npm et les GitHub Actions. Dependabot est configuré pour ouvrir des propositions hebdomadaires ; le lockfile assure la reproductibilité. Une mise à jour n'est intégrée qu'après qualification, contrôles automatisés et validation ciblée.

Étape	Action	Preuve / critère de sortie
1. Détecter	Consulter Dependabot ou lancer pnpm outdated et pnpm audit.	Mise à jour qualifiée.
2. Évaluer	Identifier patch, minor, major, sécurité, runtime ou build.	Impact et rollback définis.
3. Isoler	Traiter dans une branche dédiée avec le lockfile.	Changement traçable.
4. Vérifier	Lint, types, tests, couverture, build et audit CI.	CI verte et aucune vulnérabilité non justifiée.
5. Déployer	Fusionner puis exécuter CD et smoke tests si nécessaire.	Service stable ou rollback.
6. Tracer	Mettre à jour le changelog pour un changement notable.	Version/Unreleased cohérent.

Rythme retenu

Type	Fréquence	Règle
Sécurité critique	Sous 24 h si exploitable	Priorité maximale, validation renforcée et rollback prêt.
Patch/minor	Hebdomadaire	Traitement après CI verte.
Major framework	Mensuelle ou lot planifié	Changelog éditeur, tests ciblés et plan de retour.
Migration DB	À chaque besoin	Branche/backup Neon avant application sensible.

Traçabilité GitHub

La pull request #10, fusionnée le 7 mai 2026, a installé l'outillage MCO. L'issue #11 documente un audit de dépendances, son diagnostic, le correctif appliqué et les contrôles de validation.

3. Supervision et alerte

Le système versionné surveille les interfaces publiques critiques. Le workflow GitHub Actions Monitoring - Production health est planifié à la minute 17 de chaque heure et peut être lancé manuellement.

Sonde	Contrat contrôlé	Finalité
API readiness	GET /health/ready ; HTTP 200 et JSON status: ready.	Vérifier que l'API se déclare prête à servir des requêtes.
Web health	GET /api/health ; HTTP 2xx et JSON status: ok.	Vérifier la disponibilité du point de santé du Web.
CD	Smoke tests après déploiement.	Détecter une régression immédiatement après livraison.
CI	Lint, types, tests, build et audit.	Prévenir la livraison d'un changement non conforme.

Paramètres réellement configurés

Paramètre	Valeur configurée	Interprétation
Cadence	17 * * * *	Une exécution horaire à :17 UTC.
Seuil HTTP	200 <= status < 300	Toute réponse non 2xx est en échec.
Contrat JSON	ready pour API ; ok pour Web	Évite de considérer un simple 200 comme suffisant.
Délai/reprise	20 s par tentative ; 2 retries ; 5 s	Tolère une instabilité brève, puis remonte l'échec.
Rapport	Artefact production-health-report, même en échec	Conserve horodatage, URLs, codes et payloads.

Les endpoints de santé sont non cacheables. Ce monitoring ne mesure pas encore les métriques métier détaillées (taux 5xx, p95 IA, coût IA) : elles sont traitées comme axes d'amélioration, non comme contrôles déjà actifs.

4. Signalement et preuves C4.1.2

Lorsqu'une sonde échoue, le workflow publie une issue GitHub Production healthcheck failed avec le label monitoring, ou commente l'issue déjà ouverte. Lors d'un passage ultérieur réussi, il commente puis ferme cette issue. Le canal de signalement déclaré est donc GitHub Actions -> artefact -> GitHub Issue.

Élément	Statut	Preuve GitHub
Workflow et issue automatique	Actifs	Workflow versionné dans <code>.github/workflows/production-health-monitor.yml</code> .
Run de production	Succès le 18/08/2026	Run 32104085103 : contrôles API et Web réussis.
Artefact de monitoring	Disponible	production-health-report créé par le même run, valide jusqu'au 16/11/2026.
Incident réel	Détecté puis rétabli	Issue #42 : API HTTP 503 le 21/07/2026, puis fermeture automatique après reprise.
Alerte test sûre	Exécutée le 28/07/2026	Runs 30348338556/30348419565 et issue de test #64 fermée.

Exécutions vérifiables

- Le run de production 32104085103 a validé les endpoints API et Web et produit l'artefact de monitoring.
- L'issue #42 trace une indisponibilité API réellement détectée, le commentaire de rétablissement et la clôture automatique.
- Le run 30348338556 a exécuté `simulate_alert` : échec volontaire, artefact téléversé et issue #64 mise à jour.
- Le run 30348419565 a exécuté `simulate_recovery` : succès et fermeture automatique de l'issue #64.
- La simulation de l'issue #64 est distincte de l'incident réel #42 et n'a provoqué aucune panne de production.

GitHub Actions et GitHub Issues constituent le canal de détection et de signalement retenu pour Alcide. La supervision externe avancée reste un axe d'amélioration.

5. Collecte et consignation des anomalies

Une anomalie peut provenir du monitoring, d'un test, de la CI/CD, des logs, d'un audit ou d'un retour utilisateur. Le processus est commun : détecter, qualifier, reproduire, analyser, prioriser, corriger ou contourner, valider, informer puis clôturer.

Étape	Informations requises	Sortie
Détection	Source, date, environnement, composant, symptômes.	Signal ou ticket créé.
Qualification	Criticité P0-P3, impact, fréquence, utilisateur concerné.	Priorité et responsable.
Reproduction	Étapes, données de test, résultat attendu/observé, logs utiles.	Bug reproductible ou statut À reproduire.
Analyse	Cause racine/hypothèse et options de correction/contournement.	Décision technique argumentée.
Validation	Tests, CI, smoke test et lien correctif/PR si disponibles.	Statut résolu ou suivi planifié.

Outil de collecte

Le formulaire GitHub Anomalie Bloc 4 impose la source, l'environnement, le composant, la criticité, la description, les étapes de reproduction, l'impact, la cause, le correctif et la validation. Les fiches BUG-001 et BUG-002 conservent également l'historique de cas rencontrés pendant le projet.

L'issue #11 fournit un exemple daté de consignation : source audit sécurité, environnement, criticité P1, reproduction, impact, cause racine, correctif et validations.

6. Exemple de traitement : BUG-002

BUG-002 traite un README encodé en UTF-16 LE, rendant son affichage GitHub illisible. Ce cas illustre une anomalie de documentation réellement identifiée pendant le projet, sans revendiquer un incident de production.

Rubrique	Constat et traitement
Impact	README difficile à lire dans GitHub, mauvaise compréhension du projet et des consignes de maintenance.
Détection	Revue finale : caractères parasites et octets nuls visibles entre les caractères.
Reproduction	Lire le fichier dans un outil affichant l'encodage ou inspecter les premiers octets.
Cause	Création/copie Windows ayant conservé un encodage UTF-16 au lieu d'UTF-8.
Correctif	Réécriture en UTF-8 sans BOM et recommandation de règles .gitattributes.
Validation	Contrôle des octets et lecture GitHub/terminal attendue sans caractères parasites.
Prévention	Vérifier l'encodage des documents avant commit et utiliser un format texte UTF-8.

Passage CI/CD

Pour tout correctif applicatif, la checklist de PR impose lint, typecheck, tests, couverture, build, healthchecks si nécessaire, mise à jour du changelog et documentation du rollback. La CI est le garde-fou avant déploiement ; le smoke test production n'est revendiqué que lorsqu'il est joint.

7. Journal des versions et traçabilité

Le CHANGELOG suit Keep a Changelog et utilise les rubriques Added, Changed, Fixed et Security. La version de référence 0.13.0-rc.8 est exposée par les healthchecks API et Web de production.

Version	Date	Éléments maintenus
0.13.0-rc.8	23/07/2026	Simplification de l'interface et retrait des informations techniques IA côté utilisateur.
0.13.0-rc.7	23/07/2026	Quota jury persistant, réservation atomique et refus HTTP 429 au-delà de la limite.
0.13.0-rc.5	22/07/2026	Correctifs accessibilité, dépendances, tests, CI/CD et healthchecks de production.
0.13.0-rc.2	20/07/2026	Readiness API, tests, monitoring GitHub, templates Bloc 4 et healthchecks non cacheables.

Chaîne obligatoire pour chaque maintenance notable

- Ticket anomalie ou cas support, avec statut réel ou simulation déclarée.
- Décision de correction, contournement ou rollback, puis PR/commit lorsqu'il existe.
- Validation technique : commande, CI, test manuel et test de production seulement s'ils ont été exécutés.
- Entrée CHANGELOG et version/Unreleased correspondant au statut réellement livré.

La matrice C4.3.2 en annexe relie ces éléments et empêche de confondre un brouillon, un correctif fusionné, une validation locale et une livraison de production.

8. Support client et continuité de service

L'issue support #12, datée du 7 mai 2026, documente une mise en situation utilisateur sur la durée de génération d'un programme de huit semaines. Elle structure le contexte, le diagnostic, la contribution technique, les rôles, la résolution et la validation.

Rubrique	Éléments du cas support #12	Trace
Signal	Attente perçue comme longue ou incertaine pendant la génération IA.	Issue GitHub datée et qualifiée type:support.
Diagnostic	Parcours, logs de durée/tentatives, fournisseur, timeout, retry et validation Zod.	Contrôles techniques décrits dans l'issue.
Contributions	Utilisateur, support, mainteneur et commanditaire avec responsabilités distinctes.	Rôles et échanges consignés.
Validation	Monitoring, CI, audit corrigé et recommandations de communication utilisateur.	Liens de validation associés à l'issue.

Rollback et reprise

Situation	Réponse prévue	Preuve utile
Régression Web/API	Promouvoir le dernier déploiement Vercel sain ou appliquer un hotfix après qualification.	Capture déploiement sain/rollback et smoke test.
Migration DB risquée	Préparer une branche ou sauvegarde Neon avant migration ; privilégier des migrations compatibles.	Capture branche/backup avant exécution.
Incident de disponibilité	Consulter le rapport de monitoring, qualifier l'impact, corriger/contourner, puis valider la reprise.	Issue, artefact et test de rétablissement.

Le rollback Vercel est documenté. Pour une migration sensible, la stratégie base de données repose sur une branche ou sauvegarde Neon préalable, une migration compatible et une validation ciblée après exécution.

9. Recommandations prioritisées

Priorité	Recommandation	Gain / effort
Haute	Traiter les avis de sécurité par lots courts et maintenir l'audit bloquant dans la CI.	Réduit l'exposition et garde les mises à jour traçables.
Haute	Ajouter une notification externe indépendante pour les incidents critiques.	Réduit le délai de prise en compte ; effort faible.
Haute	Automatiser la vérification d'une branche ou sauvegarde Neon avant migration sensible.	Renforce la reprise après incident DB.
Moyenne	Centraliser les logs avec un logger structuré et des alertes 5xx/DB/IA.	Diagnostic plus rapide ; 1 à 2 jours estimés.
Moyenne	Ajouter une observabilité IA : latence, erreurs, coût et quotas.	Pilotage de la dépendance fournisseur ; 1 à 2 jours.
Moyenne	Ajouter des tests d'intégration DB dédiés.	Couvre repositories et migrations ; 2 à 3 jours.

Ces axes complètent le dispositif actuel sans remplacer les contrôles déjà actifs : healthchecks, monitoring horaire, artefacts, issues automatiques, CI/CD et procédures de rollback.

10. Grille de conformité du livrable

Le règlement attend huit éléments dans le dossier écrit. Cette grille relie chaque réponse Alcide à une preuve versionnée ou à une trace GitHub datée.

Attendu officiel	Emplacement dans ce dossier	Preuve factuelle
Mises à jour des dépendances	Section 2	Annexes A5 et A7 : issue #11, PR #10 et audit CI.
Système de supervision	Sections 3 et 4	Annexes A2 à A4 : run réel, artefact, incident #42 et test #64.
Collecte des anomalies	Section 5	Annexes A3 et A5 : issues structurées, processus et fiches BUG.
Fiche anomalie	Section 6	Annexe A5 : BUG-002, impact, cause, correctif et validation.
Traitement d'une anomalie	Section 6	Annexes A5 et A7 : correction, non-régression et CI.
Recommandations argumentées	Section 9	Annexe A1 : correspondance avec les risques et preuves disponibles.
Journal des versions	Section 7 et CHANGELOG	Annexe A8 : versions rc.2 à rc.8 et carte des fichiers.
Problème support résolu	Section 8	Annexe A6 : cas support #12, diagnostic et validation.

Traçabilité

Les références GitHub permettent de vérifier la date, le statut et l'historique des exécutions. Aucun secret, donnée de santé, adresse e-mail ou lien signé n'est reproduit dans ce dossier.

11. Routine d'exploitation proposée

La maintenance doit être pilotée dans le temps. La routine suivante est adaptée à un pilote applicatif tenu par un mainteneur principal, avec l'appui ponctuel d'un commanditaire ou d'un utilisateur pilote.

Cadence	Activité	Trace à conserver
À chaque alerte	Qualifier le rapport, mesurer l'impact, ouvrir/mettre à jour le ticket et décider correctif, contournement ou rollback.	Issue GitHub, artefact, décision et test de reprise.
Hebdomadaire	Consulter Dependabot, le résultat d'audit, les erreurs significatives et le backlog d'anomalies.	PR, résultat audit, revue des tickets ouverts.
Avant déploiement	Vérifier la CI, le changelog, le plan de rollback et les migrations éventuelles.	Run CI, PR checklist, plan de retour.
Après déploiement	Contrôler readiness API, healthcheck Web et parcours métier ciblé lorsque nécessaire.	Smoke test, run CD ou capture datée.
Mensuel	Revoir les tendances, les recommandations, les coûts/erreurs IA et la dette de maintenance.	Compte rendu, priorisation et décisions.

Rôles

Rôle	Responsabilité	Limite assumée
Mainteneur Alcide	Surveillance, qualification, correction, déploiement et documentation.	Projet individuel : pas d'astreinte multi-équipe démontrée.
Utilisateur pilote / commanditaire	Remonter un problème, valider un contournement ou un résultat métier.	Mise en situation documentée dans l'issue support #12.
Fournisseurs	Vercel, Neon, OAuth et IA : disponibilité des services externes.	Leur statut est diagnostiqué, non maîtrisé par le code Alcide.

12. Indicateurs, seuils et décisions

Les seuils suivants servent à interpréter les données de maintenance. Seuls les contrôles explicitement marqués Configurés sont automatisés aujourd'hui ; les autres sont des objectifs de pilotage à mettre en oeuvre progressivement.

Indicateur	Seuil / objectif	Statut et décision associée
Readiness API	HTTP 200 et status ready	Configuré : issue monitoring en cas d'échec.
Healthcheck Web	HTTP 2xx et status ok	Configuré : issue monitoring en cas d'échec.
Délai de sonde	20 s, 2 retries	Configuré : limiter les faux positifs liés à une instabilité brève.
Disponibilité	99 % mensuel pour le pilote	Objectif : analyser les incidents et la durée de rétablissement.
Erreurs API 5xx	< 1 % des requêtes	Objectif : centraliser les logs et alerter si le seuil est dépassé.
Erreurs IA	< 5 % des générations	Objectif : suivre fournisseur, validation et timeouts.
Latence IA	p95 < 55 s	Objectif : diagnostiquer timeout, coût ou dégradation fournisseur.

Interprétation

- Une indisponibilité Web/API constatée par le workflow déclenche la qualification d'incident et peut justifier un rollback.
- Une dérive d'erreurs IA ou de latence ne vaut pas incident prouvé tant qu'elle n'est pas mesurée ; elle est donc présentée comme une recommandation de supervision avancée.
- Les seuils métier sont révisés après collecte de données réelles du pilote, sans promettre une disponibilité contractuelle non établie.

13. Registre des risques MCO

Le registre relie les risques de maintien en condition opérationnelle aux mécanismes de détection et de réponse. Il sert de base pour prioriser les actions et documenter les arbitrages de maintenance.

Risque	Détection	Réponse et preuve
API ou Web indisponible	Readiness/healthcheck horaire, CI/CD, logs Vercel.	Issue monitoring, qualification, rollback ou hotfix, test de reprise.
Migration DB incompatible	Échec migration, erreurs de connexion ou recette ciblée.	Branche/backup avant migration, correction compatible et validation.
Régression de dépendance	Dependabot, audit, CI et tests ciblés.	Branche dédiée, lockfile, CI, rollback package/version si nécessaire.
Fournisseur IA lent/indisponible	Logs de durée, timeout, validation de réponse et retour support.	Message utilisateur, retry/backoff, analyse fournisseur et recommandation.
Erreur d'usage ou documentation	Retour support, test utilisateur, revue documentaire.	Ticket, clarification, correctif ou mise à jour de documentation.
Exposition de données sensibles	Revue sécurité, audit, logs et contrôle des captures.	Ne pas déposer de secret ; traiter et documenter toute anomalie confirmée.

Principe d'escalade

P0 : indisponibilité ou risque sécurité majeur, correction/rollback immédiat. P1 : impact élevé, traitement dans la journée. P2 : correction planifiée au prochain lot. P3 : amélioration ou dette à prioriser.

Les niveaux P0 à P3 sont des objectifs internes de priorisation. Ils ne constituent pas un SLA contractuel envers un client.

14. Synthèse des preuves et conclusion

Où trouver les annexes
Toutes les preuves sont regroupées dans le fichier annexes-bloc4-rncp39583-alcide-final.pdf. Commencer par l'index A1, puis suivre les renvois A2 à A8 de ce dossier.

Preuves GitHub vérifiées

- Annexe A2 : run de production 32104085103 réussi le 18 août 2026, payloads API/Web et artefact production-health-report.
- Annexe A3 : issue #42, indisponibilité API réellement détectée, rétablissement et clôture automatique.
- Annexe A4 : issue #64 et runs 30348338556/30348419565, simulation sans impact sur la production.
- Annexes A5 et A6 : anomalies structurées, fiches BUG et cas support pilote déclaré.
- Annexes A7 et A8 : PR/CI, CHANGELOG rc.2 à rc.8 et carte des fichiers versionnés.

Conclusion

Alcide possède une base MCO versionnée et crédible : dépendances surveillées, CI/CD, readiness, healthchecks non cacheables, monitoring horaire, rapport d'exécution, circuit d'issue, processus d'anomalies, correctifs documentés, changelog et rollback Vercel.

Les trois compétences éliminatoires sont adressées par des mécanismes concrets et des traces vérifiables : supervision et signalement C4.1.2, collecte structurée des anomalies C4.2.1 et journal des versions C4.3.2.

Référence	Objet	État
A2 - Run 32104085103	Supervision réelle API/Web et artefact	Succès
A3 - Issue #42	Incident production et rétablissement	Fermée
A4 - Issue #64	Simulation sûre du circuit d'alerte	Fermée
A5/A6 - Issues #11/#12	Anomalie dépendances et support pilote	Tracées
A7/A8 - PR #10/CHANGELOG	Outillage MCO et versions	Fusionné/versionné